SDDV

CRIPTOGRAFIA | UNAM | FI

Entrega D4

Barrios Aguilar Dulce Michelle

Contreras Colmenero Emilio Sebastian

Martinez Lopez Evan Emiliano

Caballero Martinez Sergio Jair

Boveda Digital

Segura de Documentos

Cifrado AEAD | Firmas Ed25519 | Cifrado Hibrido X25519

4	966	94
Modulos	Lineas	Tests

Dr. Rocio Aldeco Perez

A System Overview

Que hace el sistema y que problema resuelve — 1–2 min

El problema

> Los archivos digitales se almacenan sin cifrado.

> Un atacante con acceso al disco puede leerlos o modificarlos.

> No hay forma de saber si un archivo fue alterado.

> Compartir archivos cifrados con varias personas es complejo.

->

SDDV — la solucion

> Biblioteca Python de cifrado autenticado (AEAD).

> Cada archivo queda sellado: confidencial e integro.

> Firmado digitalmente: autoria verificable.

> Cifrado hibrido: compartible con multiples destinatarios.

Confidencialidad

AES-256-GCM
ChaCha20-Poly1305

Integridad

TAG 128 bits
detecta cualquier cambio

Autenticidad

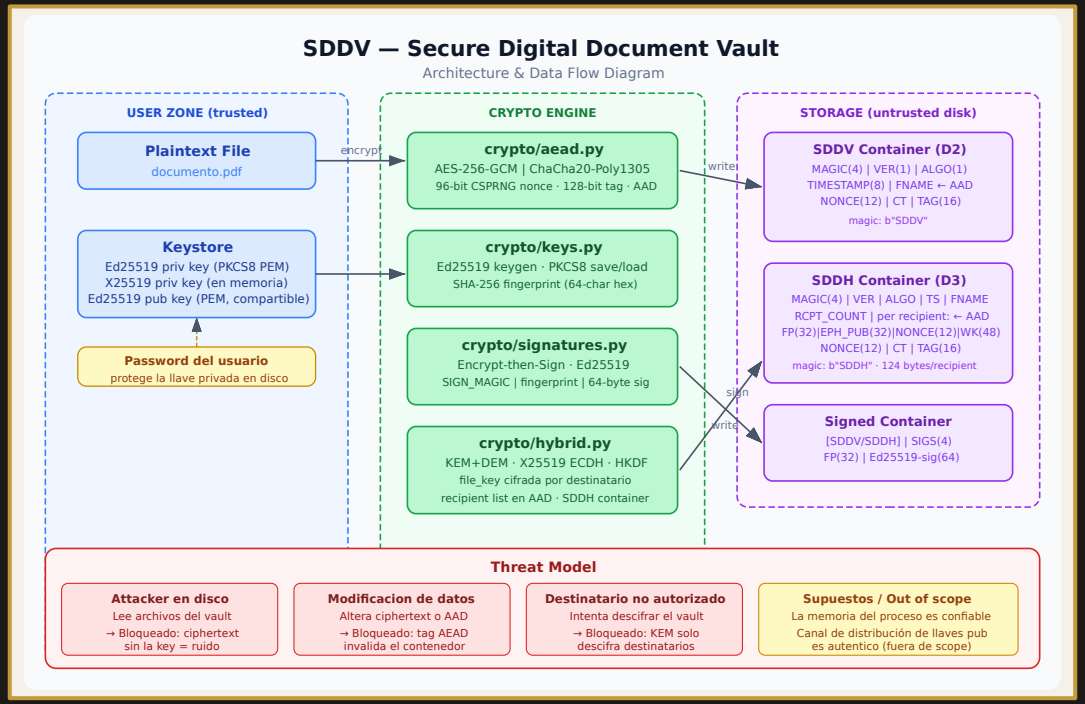
Firma Ed25519
verifica el emisor

Containers producidos:

SDDV — cifrado simetrico (un destinatario) | SDDH — cifrado hibrido (multiples destinatarios) | +SIGS — contenedor firmado

B Arquitectura del Sistema

Componentes, flujo de datos y limites de confianza — 2–3 min



Flujo: Plaintext > encrypt_file() > Container SDDV > sign_container() > Container Firmado > Disco / Red

Componentes

Vault (contenedor)

SDDV / SDDH — archivo cifrado en disco

Keystore

Llaves Ed25519 + X25519 en PEM PKCS8

Motor AEAD

aead.py — cifra y autentica el archivo

Modulo Firmas

signatures.py — Encrypt-then-Sign

Limites de confianza

Confiable: proceso en RAM, llaves del usuario

No confiable: disco, red, servidor de archivos

C Modelo de Amenazas

Atacante, activos protegidos y suposiciones del sistema — 2–3 min

Atacante

! Acceso fisico al disco

! Captura pasiva de red

! Modificacion activa de archivos

! Destinatario no autorizado

! Servidor de almacenamiento comprometido

Activos protegidos

Contenido del archivo

Cifrado con AEAD 256 bits

Identidad del emisor

Firma Ed25519 verificable

Lista de destinatarios

En AAD, protegida por TAG

Llave privada

PEM PKCS8, cifrada en disco

Integridad de metadatos

Filename y timestamp en AAD

Suposiciones

> **RAM es confiable**

El proceso en ejecucion no esta comprometido

> **Llaves privadas son secretas**

El usuario protege su contrasena PEM

> **Canal de distribucion de llaves publicas esta fuera del scope**

> **La implementacion de cryptography es correcta**

Usamos la biblioteca estandar de Python

D Diseno Criptografico — D2: AEAD

Eleccion AEAD, estrategia de nonce y metadatos + AAD — (3–4 min total con D3)

Eleccion AEAD

AES-256-GCM

> Estandar NIST, acelerado por hardware

> 256 bits de clave — 128 bits de seguridad

> TAG de 128 bits sobre ciphertext + AAD

ChaCha20-Poly1305

> Igualmente seguro, eficiente en software

> Preferible sin aceleracion hardware

> Misma interfaz, alternativo al usuario

Estrategia de Nonce

96 bits = 12 bytes

Generado con os.urandom()

CSPRNG del sistema operativo

Unico por operacion de cifrado

Nunca se reutiliza la misma (clave, nonce)

Almacenado en el contenedor

El receptor no necesita canal adicional

Colision despreciable

2^48 cifrados para P=10^-6

Metadatos + AAD

AAD = datos autenticados
pero NO cifrados

Incluido en el AAD:

- **Filename**
  No se puede renombrar el contenedor
- **Timestamp Unix**
  No se puede copiar el tiempo
- **Algoritmo (ALGO_ID)**
  No se puede cambiar el algoritmo
- **Lista de destinatarios**
  (SDDH) Integridad del acceso

Si cualquier byte del AAD cambia, el TAG falla al verificar.

D Diseno Criptografico — D3: Cifrado Hibrido

Flujo KEM+DEM, manejo de destinatarios e identificacion de llaves

Flujo KEM+DEM

DEM — Data Encapsulation Mechanism

1. Generar file_key aleatoria de 256 bits (CSPRNG)
2. Cifrar el archivo con AES-256-GCM usando file_key
3. El AAD incluye la lista completa de destinatarios

KEM — Key Encapsulation Mechanism

1. Por cada destinatario: generar par X25519 efimero
2. ECDH(eph_priv, dest_pub) -> secreto de 32 B
3. HKDF-SHA256(secreto, salt=fingerprint) -> wrapping_key
4. AESGCM(wrapping_key).encrypt(nonce, file_key) -> 48 B

Destinatarios

Lista en AAD

Cualquier cambio invalida el TAG

Slot fijo 124 B

FP+EPH_PUB+ NONCE+KEY

Solo titular

Solo quien tiene la llave privada

Forward Secrecy

Par efimero unico por destinatario

Identif. de Llaves

Fingerprint

SHA-256 del raw public key = 64 hex chars

En el container SDDH

Cada slot guarda el FP del receptor para que pueda buscarse sin leer el resto.

Un receptor busca su FP -> descifra su slot -> obtiene

Contenedor SDDH:

MAGIC	HEADER	FINGERPRINT	EPH_PUB	WRAP NONCE	WRAPPED KEY	x N dest.	CT + TAG

E Demo en Vivo

Cifrar, compartir, descifrar y verificar fallos — 2–3 min | python demo.py

1 Cifrar un archivo

encrypt_file() con AES-256-GCM — genera contenedor SDDV

OK Contenedor SDDV creado

2 Compartir con otro usuario

encrypt_for_recipients([alice_pub, bob_pub]) — genera contenedor SDDH

OK Alice y Bob como destinatarios

3 Descifrar como usuario autorizado

decrypt_for_recipient(container, alice_priv) — Alice descifra

OK Alice descifra | OK Bob descifra

4 Fallo: no autorizado y tamper

Eve intenta descifrar | se modifica 1 byte del ciphertext

FALLA InvalidTag en ambos casos

Ejecutar: python demo.py

Gracias

Boveda Digital Segura de Documentos | SDDV

A

AEAD AES-256-GCM + ChaCha20

Confidencialidad e integridad

B

Ed25519 + Encrypt-then-Sign

Autenticidad del emisor

C

KEM+DEM X25519 + HKDF

Cifrado hibrido multi-dest.

D

94 tests | 0 fallos

966 lineas | 4 modulos

Dr. Rocio Aldeco Perez | Criptografia | UNAM 2026